

# eGPU: Production-Scale Elastic Sharing over 10,000 GPUs

Xiaochuan Tang<sup>1†</sup>, Hao Qi<sup>2†</sup>, Jianbo Dong<sup>1</sup>, Yinghao Yu<sup>1</sup>, Zhennan Xue<sup>1</sup>, Zhengyu Zhang<sup>1</sup>,  
Daocheng Ying<sup>1</sup>, Zheng Cao<sup>1</sup>, Xiaoyi Lu<sup>2\*</sup>

<sup>1</sup>Alibaba Group

<sup>2</sup>University of California, Merced

<sup>†</sup>Co-first authors    <sup>\*</sup>Corresponding author

{xiaochuan.txc, jianbo.djb, yinghao.yyh, xuezhennan.xzn, zhangzhengyu.zzy,  
daocheng.ydc, zhengzhi.cz}@alibaba-inc.com  
{hqi6, xiaoyi.lu}@ucmerced.edu

**Abstract**—As the cost of GPUs continues to rise, GPU-sharing solutions have become increasingly important for improving efficiency and maximizing resource utilization. At the same time, large-scale operational deployments of such solutions remain relatively less explored, especially in heterogeneous production environments where workload dynamics and orchestration complexity introduce new practical considerations. In this paper, we introduce eGPU, an elastic, efficient, and scalable GPU-sharing framework tailored for production-scale concurrent machine learning (ML) training and inference. eGPU enables fine-grained, runtime-adjustable sharing of GPUs across multiple jobs, while preserving high resource utilization and fault isolation. To address communication bottlenecks, eGPU supports native NVLink/NCCL-based communication between shared GPU instances, capabilities that are limited or unavailable in many existing designs. Built with production deployment in mind, eGPU integrates with Kubernetes (K8s) to support large-scale orchestration. It has been deployed and running stably in production clusters with over 10,000 GPUs for five years. Our evaluation results show that eGPU achieves elastic and precise control over instance sizes, improves job efficiency by 21% to 31% than SOTA sharing solutions, saves the number of GPUs required by up to 8×, and improves cluster GPU utilization by more than 3×.

## I. INTRODUCTION

GPUs have become essential accelerators for numerous applications, such as machine learning (ML) [31], [47], scientific simulations [28], [41], and data analytics [44], [54]. Their continuous evolution in performance and efficiency [37], [38] has significantly enhanced computational capabilities. However, GPUs are expensive resources, and not all HPC and ML jobs have high GPU demands, making their utilization inefficient when dedicated to a single application [16], [26], [45]. For distributed jobs, it even causes utilization to go lower [27]. Recent studies further highlight that GPU resources in large-scale data centers frequently remain underutilized, exacerbating resource wastage and operational inefficiencies [26], [55], [58].

These observations have driven the development of GPU-sharing solutions, enabling multiple concurrent applications to efficiently utilize GPU resources. GPU sharing is particularly beneficial in environments like institutional clusters and enterprise clouds, where resource optimization directly

translates into cost savings and improved sustainability. From a cluster perspective, GPU sharing minimizes resource wastage, enhances cluster-wide utilization, and supports sustainable computing practices. From a user perspective, it reduces costs by enabling payment solely for their actual GPU usage rather than entire dedicated GPUs.

Existing GPU-sharing solutions generally fall into two categories: (1) *spatial sharing* and (2) *temporal sharing*. Spatial sharing approaches [7], [8], [17], [22], [48] partition GPU resources, such as memory and cores, allowing simultaneous usage by multiple applications to maximize resource efficiency. Temporal sharing approaches [21], [56]–[59] allocate GPU resources sequentially and exclusively across different time slots, reducing idle periods and enhancing overall efficiency. Despite considerable progress, our experience shows that SOTA solutions face challenges when we attempt to adopt them in real-world production environments (Sec. III). Through this process, we identify six challenges that are either unique to production or become significantly more severe at scale, often due to fundamental design assumptions underlying existing solutions. Unfortunately, and somewhat surprisingly, we found that no single solution was able to overcome all six challenges simultaneously. We categorize these challenges into three crucial areas: **elasticity**, **efficiency**, and **scalability**.

①Elasticity involves precisely configuring GPU instances according to user requirements and dynamically adjusting resource allocations at runtime. In production clusters, workloads exhibit highly dynamic and unpredictable demands, requiring flexible resource management (Sec. II-C) [30], [55]. However, many existing solutions [22], [39], [40], [48] lack sufficiently fine-grained resource allocation capabilities and cannot dynamically adjust resource ratios between instances at runtime (P1 in Sec. III-B). One industry-provided solution, Multi-Process Service (MPS) [8], offers runtime flexibility, but at the price of shared failure domains: an error in one job will terminate all other jobs sharing the GPU (P2 in Sec. III-B).

②Efficiency challenge lies in guaranteeing user Service Level Agreements (SLAs) without compromising job integrity, i.e., modifying batch sizes or the number of GPUs (P3 in

Sec. III-B). In production clusters, SLA compliance is critical: High GPU utilization is of little value if it results in SLA violations. Several earlier works often introduce overheads due to the added virtualization layer [42], [46], [49]. Priority-based approaches [17], [22], [56]–[58] often run low-priority jobs opportunistically, which may unintentionally compromise SLA expectations (P4 in Sec. III-B). Their drawback is the lack of execution guarantees for low-priority jobs. We have seen users complaining about their jobs being delayed indefinitely or never running at all.

③ Scalability emphasizes maintaining user transparency, robust fault isolation, and broad applicability across heterogeneous GPUs in large-scale production clusters. During our exploration, we find solutions demanding extensive modifications from users [58], [59] (P5 in Sec. III-B) or intrusive GPU kernel management [21], [22] are challenging to adopt, as there may be thousands of diverse users and closed-source hardware in production clusters (P6 in Sec. III-B). Fault isolation also remains a critical concern, as merging kernels from different jobs, as in MPS-based methods [8], [17], risks the stability of all tasks running concurrently on the GPU.

Motivated by these operational challenges, we rethink SOTA solutions (Sec. II-C) and present our answer: **eGPU** (Sec. IV), an elastic GPU-sharing solution designed specifically for production-scale ML training and inference workloads. ***eGPU has been successfully deployed and has operated stably for five years in production clusters comprising over 10,000 heterogeneous GPUs.***

eGPU achieves ① elasticity by proposing a temporal sharing technique called the *Fine-grained Anti-starving Algorithm (FAA)* (Sec. IV-B), leveraging CUDA/HIP API interception. Unlike traditional kernel-based approaches [22], [48], FAA divides the allocation window into shorter, dynamically scheduled time slices, effectively preventing resource starvation and enabling precise runtime adjustments. eGPU achieves ② efficiency through optimizing inter-instance communication (Sec. IV-D), addressing a critical but often overlooked performance bottleneck. When possible, eGPU supports native NVLink and NCCL communication among shared instances, ensuring efficient data transfers without user-visible complexities. ③ eGPU’s scalable instance control integrates seamlessly with Kubernetes (K8s), overcoming the platform’s default limitations for GPU partitioning and runtime adjustment (Sec. IV-E).

Our major contributions are summarized as follows:

- We summarize unique challenges brought by production environments for GPU sharing, which prevent existing solutions from fitting well. Meanwhile, we share our lessons learned and present our production-ready solution eGPU, by overcoming challenges in elasticity, efficiency, and scalability.
- We evaluate eGPU with extensive experiments from various aspects. eGPU shows significant benefits for the designed objectives. In particular, eGPU can improve job efficiency by 21% to 31% than SOTA sharing solutions, save GPU resources by up to 8 $\times$ , and improve cluster GPU

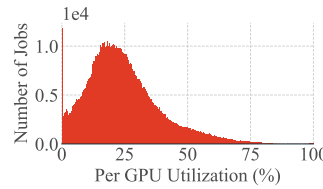


Fig. 1: GPU Utilization Distribution of Cluster User ML Jobs.

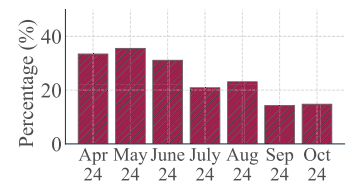


Fig. 2: Percentage of Jobs with Priority-based Sharing in a Production Cluster.

utilization by more than 3 $\times$ .

- We have deployed eGPU at production-scale data center clusters with more than 10,000 GPUs. eGPU has been providing a stable functional performance for five years and is still serving continuously and daily.

***To the best of our knowledge, this is the first disclosed production-ready GPU-sharing solution for datacenter-scale clusters.***

## II. BACKGROUND

### A. Opportunities of GPU Sharing

A key aspect of GPU sharing is the fact that not all workloads demand the full capacity of a GPU. For instance, a considerable number of ML tasks are not computationally demanding and can run smoothly on low-end GPUs or a small portion of high-performance GPUs. Another motivation for GPU sharing is the steady increase in GPU potency [5], [6], [34]. GPUs are expensive and the hardware resources of a single GPU have continued to escalate at a rapid pace, allowing a single GPU to handle multiple smaller workloads concurrently without compromising performance.

Moreover, recent strides in virtualization methods have made it easier to share GPUs among multiple users or tasks [20], [24], [43]. For example, the cloud-native business platform running with containerization and K8s scheduling has become the trend and direction for data center operations in the industry [33], [52]. In many cases, an ML task may not fully utilize the compute and/or memory resources of each entire GPU it requests, and some online inference containers of certain users run continuously at low levels for prolonged periods of time. To illustrate this point, we present statistics on per GPU utilization<sup>1</sup> gathered from approximately one million user jobs on an internal production cluster in two months in Figure 1. Numbers are reported on a per-GPU basis. E.g., a distributed job may allocate 4 GPUs but only achieve an average utilization of 40% on each GPU. The findings show that a majority (around 93%) of the workloads used less than 50% of the GPU, and the mean GPU utilization of all jobs is merely 24.70%. These situations result in a significant waste of GPU resources.

<sup>1</sup>We are following the definition from NVIDIA: percent of time over the past monitor period during which at least one kernel is executing.

## B. Existing Sharing Solutions

We next review some representative GPU-sharing solutions. **Temporal Sharing.** Gandiva [57] exploits a suspend-resume mechanism to switch ML application states between CPU and GPU memory due to their cyclic behavior. Antman [58] controls the GPU operators by a manager and dynamically grows or shrinks job memory allocation for sharing. Clockwork [21] predicts the inference time by a consolidating choice approach and can share thousands of inference tasks per GPU. TGS [56] uses an adaptive rate control algorithm to adjust resources for opportunistic jobs based on high-priority jobs' kernel arrival rate. Salus [59] proposes two primitives, fast job switching and memory sharing, to context switch and pack jobs.

**Spatial Sharing.** REEF [22] co-executes jobs with a granularity of kernels and preempts best-effort job kernels when the GPU resource is needed by real-time jobs. Orion [48] classifies jobs into memory-intensive or compute-intensive and collocates one from each type to improve GPU utilization. GPUlet [17] aims to find the most cost-effective arrangement from a cluster perspective by adjusting parameters such as batch size for each job. MPS [8] works by allowing multiple ML applications to share a single GPU context, enabling concurrent execution. MIG [7] partitions GPU hardware into multiple isolated GPU instances, each with its own dedicated memory and compute resources, allowing for guaranteed performance isolation.

## C. Rethinking SOTA Solutions

**Priority-based sharing vs. Partition-based sharing.** Many existing GPU-sharing approaches adopt a priority-based strategy [22], [48], [56], where GPU resources are allocated based on job priority. These approaches typically assume that jobs can be clearly categorized into high- and low-priority classes, allowing for opportunistic scheduling of lower-priority jobs when leftover resources are available.

However, we find priority-based sharing in production environments presents practical challenges. These approaches typically rely on users to classify their jobs into high and low priority. In practice, we observed that users tend to label their jobs as high priority, which results in a scarcity of low-priority jobs available to share resources. Moreover, priority-based approaches offer no guarantee of execution for low-priority jobs. We have received reports from users whose low-priority jobs remained pending for over a week without being scheduled, raising concerns about fairness and predictability.

In contrast, partition-based sharing solutions [7], [8] allow users to request specific partition sizes based on their workload demands, treating all jobs with no priorities. Figure 2 presents data collected over a six-month period, from April 2024 to October 2024, from our production cluster. It illustrates the percentage of GPUs where users opted for priority-based sharing. The results clearly indicate that priority-based sharing is less favored in production environments, with usage ranging between 14.3% and 35.5% across different months.

Therefore, we make eGPU a design (details in Sec. IV) that is primarily partition-based but can also support priority-based

functionality when needed. By dynamically resizing partitions, eGPU can adjust resources to prioritize certain jobs while still providing resources for lower-priority tasks, achieving the same function as priority-based solutions.

**Temporal sharing vs. Spatial sharing.** Spatial sharing presents an intriguing direction worth exploring, as jobs can access different portions of the GPU simultaneously, typically resulting in higher GPU utilization than temporal sharing.

However, creating production-ready spatial sharing solutions still necessitates addressing flexibility and fault isolation concerns. Solutions [8], [22], [48] that manage kernels face fault isolation concerns. For instance, MPS [8] allows multiple jobs to share a single GPU context, which can lead to cascading failures—an error in one job may disrupt all co-located jobs. This limitation extends to MPS-based systems such as GPUlet [17]. On the other hand, solutions that offer strong fault isolation, like MIG [7] and TGS [56], pay the price of performance. Inter-process communication is restricted, and there is no NVLink support, due to their design limitation. Moreover, MIG provides poor flexibility, as all MIG instances need to be preconfigured with fixed sizes, and no runtime adjustments are available.

Considering these trade-offs, eGPU is built around a temporal sharing design (details in Sec. IV). Nonetheless, we recognize the promise of spatial sharing and ensure eGPU is compatible with such solutions. This opens the door for spatial-temporal sharing. For example, eGPU can complement MPS in trusted environments where fault isolation is less critical, or work alongside MIG in use cases where performance trade-offs are acceptable in exchange for strong isolation.

## III. MOTIVATION AND CHALLENGES

### A. Motivation

**Need for Elasticity from ML Workloads.** Machine learning jobs often require dynamic and diverse resource allocation, making elasticity a key factor in optimizing GPU use. For example, in real-time recommendation systems, such as those used in online shopping, demand for inference tasks surges during peak hours and drops significantly at night. To handle these fluctuations, GPU-sharing mechanisms must be capable of adapting resource allocations dynamically.

Existing GPU-sharing solutions are very limited in this regard. Many provide only static partitioning or lack fine-grained control over instance sizes, preventing runtime adjustments. For instance, MIG [7] requires fixed partition sizes to be preconfigured (e.g., 1/7 of a GPU). If a user's workload slightly exceeds the assigned size, it cannot be accommodated without using a larger partition. Meanwhile, many priority-based designs [22], [56], [58] allow only low-priority jobs to opportunistically utilize spare resources and do not provide users with explicit control over GPU partition sizes.

**Need for Efficient Communication during Sharing.** As machine learning models grow increasingly complex and data-intensive, ensuring communication efficiency becomes essential in shared GPU environments. Prior studies [32],

[50], [53] have shown that communication, not just computation, can become a major bottleneck in large-scale ML workloads. To improve communication efficiency, high-performance GPUs commonly utilize a high-speed dedicated bus, such as NVLink [9], for communication. The fifth-generation NVLink boasts a total bandwidth of 1800 GBps, approximately 7 times that of PCIe Gen6 [11].

However, we observed that many GPU-sharing mechanisms cannot fully utilize NVLink due to resource encapsulation practices aimed at ensuring fault isolation. Specifically, when GPUs are isolated at the container or hardware level, the NVLink interconnect becomes unavailable to applications. For example, MIG [7] disables NVLink support as a consequence of its architectural isolation. Similarly, in containerized solutions [56], GPUs assigned to different pods, even if physically on the same node, are unable to communicate via NVLink due to software-level boundaries. This limits communication bandwidth and can hinder model training or inference performance.

Moreover, the widely used ML collective communication library NCCL [36] currently does not support multiple instances using the same physical GPU for the same task. The NCCL communication library document explicitly states that ‘Using the same CUDA device multiple times as different ranks of the same NCCL communicator is not supported and may lead to hangs.’ [35] In the most recent NCCL v2.28, NCCL gives a prompt message when it detects a shared device, and an invalid usage error is returned. The NCCL development team is concerned that allowing shared equipment may not be deadlock-safe or may even break in some cases.

**Need for Scalability to Production-Scale** We also find scaling GPU sharing to production-level heterogeneous clusters introduces additional challenges. Current solutions typically support a limited number of co-located instances (e.g., 7 for MIG, 5 for priority-based designs), which becomes restrictive in environments with thousands of GPUs. This limitation constrains both resource flexibility and job placement decisions.

In industry, production systems often rely on container-based isolation, with Kubernetes (K8s) [15] serving as the de facto resource orchestration layer. However, the default K8s framework lacks native support for several capabilities critical to scalable GPU sharing, most notably GPU partitioning and runtime resource adjustment. Without scheduler-level awareness of partitionable GPU resources, effective cluster-wide resource management becomes difficult.

Another problem with current solutions is the potential for resource misuse and conflicts in user tasks. Existing K8s implementations assign GPU resources using environment variables, such as `VISIBLE_DEVICES`, which opens the door for users to inadvertently access GPU resources not allocated to them, undermining resource isolation. Finally, current solutions struggle to support elastic resource adjustments during runtime. Once an instance is set up in K8s, it becomes static, preventing changes to GPU compute and memory allocation.

## B. Challenges from Production Clusters

We distill the previous discussions into six key challenges, summarized in Table I. We acknowledge that SOTA sharing solutions have made notable progress. However, broader requirements become critical in production-scale clusters. In many cases, SOTA solutions address only a subset of these challenges, as other challenges may conflict with their fundamental working principles. Therefore, addressing all six challenges simultaneously presents a much greater challenge. **P1: Achieve Allocation Elasticity.** Many existing GPU-sharing mechanisms allocate fixed resources or rely on static job priority to mediate sharing, but do not allow for precise or dynamic control over resource ratios. In production environments, this limitation becomes more evident, as ML jobs frequently experience fluctuating resource demands. Enabling users to request a partition of a GPU and flexibly adjust this partition size at runtime is an important requirement for elastic resource allocation.

**P2: Provide Fault Isolation.** In production clusters, job failures are inevitable. Systems that merge multiple jobs into a shared GPU context risk allowing failures in one job to impact all others, an unacceptable outcome in multi-tenant settings. Ensuring fault isolation, even while enabling GPU sharing, is therefore a necessary consideration in any practical deployment.

**P3: Keep Job Integrity.** Some existing solutions aim for efficiency by modifying user job configurations, such as adjusting batch sizes or modifying resource request amounts. Some others rely on pre-classifying jobs or pairing complementary workloads. In production practice, anticipating user intent or requiring such configuration changes may not be feasible. Designing a system that preserves user-specified settings and treats jobs independently without requiring classification is critical.

**P4: Guarantee SLA Compliance.** Systems based on job priority often prioritize overall resource efficiency at the cost of user service level agreement (SLA), as they typically guarantee progress only for high-priority jobs. In some cases, low-priority jobs can remain pending indefinitely. We have observed instances where jobs waited more than a week without being scheduled. A production-ready system must be able to ensure that all jobs, regardless of priority, make progress within a reasonable time frame.

**P5: Maintain User Transparency.** Framework-level solutions frequently require modifications to user code or depend on application profiling to optimize sharing. However, in many production scenarios, GPU jobs represent just one component of larger, multi-stage workflows. Extracting GPU job profiles or requesting example inputs is often impractical. A practical sharing solution must operate without imposing changes on users or requiring additional metadata from jobs.

**P6: Support GPU Heterogeneity.** Production clusters are typically composed of GPUs acquired over time, resulting in heterogeneous and often partially closed-source hardware. Many existing solutions assume specific GPU capabilities, such as full kernel visibility, which are not always available.

Supporting a diverse range of GPUs, including those without hardware partitioning or open software interfaces, is essential for wide deployment.

TABLE I: eGPU and Existing GPU-sharing Solutions

|    | Salus [59] | Clockwork [21] | Antm-an [58] | TGS [56] | REEF [22] | GPU-et [17] | Orion [48] | MPS [8] | MIG [7] | eGPU                  |
|----|------------|----------------|--------------|----------|-----------|-------------|------------|---------|---------|-----------------------|
| P1 | ×          | ×              | ×            | ×        | ×         | ✓           | ×          | ×       | ×       | ✓(FAA)                |
| P2 | ×          | ✓              | ✓            | ✓        | ×         | ×           | ✓          | ×       | ✓       | ✓(K8s Isolation)      |
| P3 | ✓          | ✓              | ×            | ×        | ×         | ×           | ×          | ✓       | ✓       | ✓(No Setting Changes) |
| P4 | ×          | ×              | ✓            | ✓        | ✓         | ✓           | ✓          | ×       | ×       | ✓(Partition Sharing)  |
| P5 | ×          | ×              | ×            | ✓        | ×         | ×           | ×          | ×       | ✓       | ✓(API Interception)   |
| P6 | ✓          | ✓              | ✓            | ✓        | ×         | ×           | ✓          | ×       | ×       | ✓(CPU-side Oper.)     |

#### IV. OUR SOLUTION: eGPU

##### A. System Overview

For complex large-scale production clusters, a GPU-sharing solution that works well in practice must be simple yet effective. Motivated by the six production challenges summarized in Section III-B, eGPU is designed for deployment in production scale clusters to improve the utilization of heterogeneous GPU resources for simultaneous use. With a primary focus on ensuring the precise partitioning of GPUs and guaranteeing performance metrics post-resource allocation, eGPU operates as a node-level GPU partitioner, orthogonal to cluster schedulers like Kubernetes (K8s) or Slurm. When a user submits a job requiring GPU resources, eGPU creates partitions on a node and dynamically adjusts these during runtime. This partitioning is seamless to users, who interact through OCI-compatible containers without needing to modify their tasks. For distributed jobs, eGPU functions at every node they are allocated with. As a result, from a user job view, there is no extra kernel scheduling order or extra communication issues. Compared to no-sharing conditions, the only difference they see is the GPU compute capability and memory capacity.

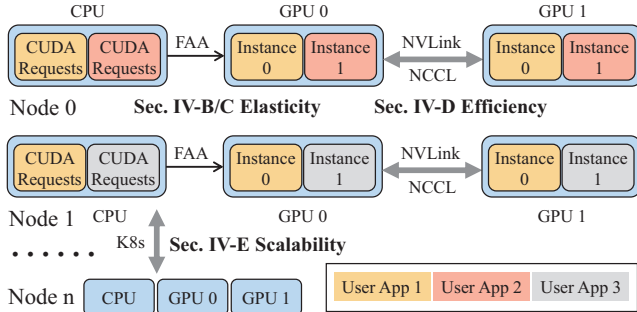


Fig. 3: eGPU Overview. eGPU ensures elasticity by adopting FAA, provides efficiency by overcoming communication bottlenecks, and achieves scalability by optimizing K8s.

The eGPU architecture overview with an illustrative example is shown in Figure 3, where each node is equipped with 2 GPUs. In this example, 1st user application is asking for four 50% GPUs and is allocated on Nodes 0-1. 2nd and 3rd user applications are asking for two 50% GPUs and are allocated on Node 0 and 1, respectively. eGPU functions purely on the CPU

side. eGPU achieves elasticity by adopting *Fine-grained Anti-starving Algorithm (FAA)* that builds upon the CUDA request intercept concept (We use NVIDIA GPUs in this paper. To function on AMD GPUs, we simply need to focus on intercepting HIP API calls). To achieve GPU compute partitioning, CUDA requests from one instance are only allowed to pass to the GPU side during its own multiple shorter time slices. As for GPU memory management, instead of memory swapping during context switches, eGPU intercepts CUDA API calls related to memory allocation. If a memory request exceeds the allocated quota, the API call is blocked and returns a failure. eGPU's efficiency is achieved by allowing all GPUs on the same node to utilize NVLink and NCCL for efficient communication, while user tasks only view the resources they have requested. eGPU's scalable instance control relies on optimizing K8s with the ability to manage partitioned GPUs and make runtime adjustments on instances.

##### B. Elastic Partitioning

In traditional temporal sharing methods, a static and inflexible percentage of resources is allocated to each instance over long monitoring periods (e.g., 1s). As an example, CUDA requests from an instance allocated 70% of the resources would be allowed to pass to GPUs only during a 0.7s time period in the past 1s, and an instance allocated 30% would do so during a 0.3s period, which can lead to stagnation and resource starvation.

We observe that **GPU kernel duration is dominated by short kernels** ( $< 0.1$  s). Under such short kernels, coarse sharing windows (e.g., 0.7s) cause transient starvation. As an example, Figure 4 shows the distribution of kernel lengths for around 20 user jobs on V100 GPUs in a production cluster. In this trace,  $> 85\%$  of GPU kernels are within 0.1ms and  $> 98\%$  are within 1ms.

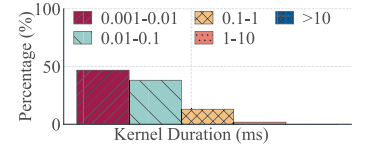


Fig. 4: Kernel Length.

Inspired by this observation, eGPU introduces a *Fine-grained Anti-starving Algorithm (FAA)*, which leverages CUDA request interception (Sec. V) and emphasizes fine-grained partitioning with anti-starvation scheduling. FAA addresses the issues above by splitting the allocated time period into multiple shorter slices (e.g., a 0.7 s share into seven 0.1 s slices) and assigning each next slice to the most under-served instance, preventing starvation and driving the allocation toward target shares, as shown in Figure 5.

The scheduling of the next slice is based on the scheduling histories in the past monitoring periods. We aim to adjust the next slice so that the ratio for each instance's slices converges towards the ratio of their requested percentages ( $70\%:30\% = 7:3 = 2.33:1$ ). For example, at time point 0.7s, we have 'abaabaa' already, then the current ratio is  $5:2 = 2.5:1$ , which is higher than 2.33:1. Therefore, we will choose the next slice as 'b' to reduce the ratio to  $5:3 = 1.66:1$ . And then the next

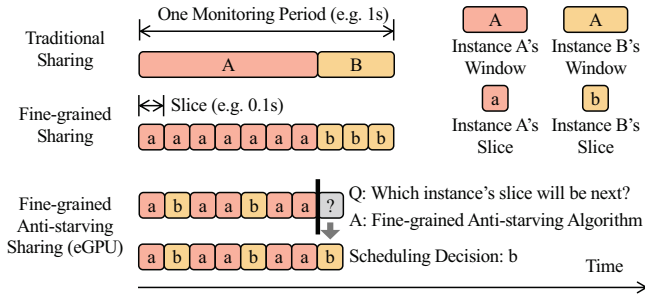


Fig. 5: eGPU Fine-grained Anti-starving Algorithm.

slice will be scheduled for 'a' to change to  $6:3 = 2:1$ . The overall process is converging the ratio towards 2.33:1. The generalized algorithm is shown in Algorithm 1.

**Algorithm 1** Fine-grained Anti-starving Algorithm (FAA)

**Input:** Instances  $I_1, I_2, \dots, I_n$  with requested percentages  $P_1, P_2, \dots, P_n$   
**Output:** Time slice allocations for each instance  
Initialize time slice list  $T = []$   
Initialize scheduling history  $H = []$   
**for** each monitoring period **do**  
  **for** each instance  $I_i$  **do**  
    Number of slices  $S_i = P_i \times \text{Total number of slices}$   
    Allocate  $S_i$  slices to  $I_i$  in  $T$   
  **end for**  
  Record allocation in  $H$   
  Calculate current allocation ratio for each instance  $R_{\text{current},i}$  based on  $H$   
  Initialize deviation array  $D = []$   
  **for**  $i = 1$  to  $n$  **do**  
    Calculate target allocation ratio  $R_{\text{target},i} = \frac{P_i}{\sum_{j=1}^n P_j}$   
    Calculate deviation  $D_i = R_{\text{current},i} - R_{\text{target},i}$   
  **end for**  
  Find instance  $I_{\text{min\_dev}}$  with minimum deviation in  $D$   
  Allocate next slice to  $I_{\text{min\_dev}}$   
  Adjust  $H$  for the next monitoring period  
**end for**  
**return**  $T$

The parameters, such as the length of the monitoring period and time slices, are key to making FAA effective. However, they are usually non-deterministic and informed by practical considerations and hardware constraints. According to our tuning, the monitoring period typically ranges from 1/6s to 1s, aligning with NVIDIA's sampling information, while the time slice length is chosen based on a balance between finer granularity and hardware limitations, usually falling between 0.01s and 0.02s.

### C. Elastic Adjustment

A key feature of eGPU is the ability to elastically adjust the resources allocated to each instance during runtime, which is crucial for optimizing GPU utilization and accommodating

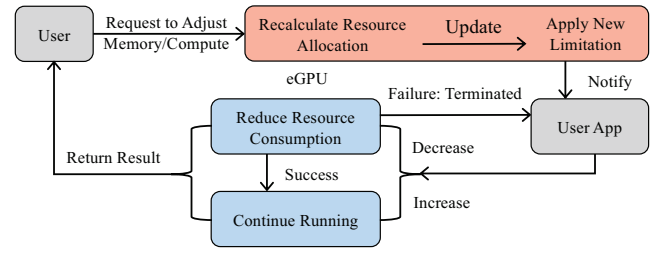


Fig. 6: eGPU Instance Resource Adjustment.

fluctuating workload requirements. Using the same example, decreasing an instance from 70% to 60% would correspond to a 0.6s time period and setting a new converging ratio as 6:3:1. Note that the value '1' here indicates extra resources left. In eGPU, a notification mechanism, as shown in Figure 6, is utilized to inform the user application of any necessary adjustments. With our FAA, this dynamic adjustment can be performed seamlessly, without interrupting the operation of the instance. The reason for adjustments can be based on various factors, such as changes in user requirements or specific policies defined by the system administrator.

Currently, instance sizing in eGPU is determined through a try-and-error process. Users can submit jobs with different instance sizes or gradually adjust their allocations based on observed job performance. This allows them to balance the trade-off between paying for larger instances and achieving the desired performance for their workloads.

### D. Efficient Communication Support

To enhance communication efficiency in multi-tenant environments, eGPU is designed to support high-speed interconnects such as NVLink for cross-instance communication within a node, as illustrated in Figure 7. Efficient communication is essential in distributed machine learning workloads, where gradient synchronization and collective operations often constitute a significant portion of training time.

To achieve this, eGPU implements a *topology-aware NVLink communication scheme*. The method proceeds in the following steps: (1) Upon receiving user specifications, eGPU instantiates the corresponding GPU partitions for each request. In this example, three such requests are issued and assigned to separate eGPU instances. (2) During instance initialization, eGPU makes all physical GPU devices visible to each container, and sets an environment variable `eGPU_NODE=nodename`. This enables each NCCL rank to determine whether its communication peers are co-located on the same physical node. (3) When launching the instances, eGPU intercepts NCCL library calls to ensure that NVLink-aware peer-to-peer communication is enabled only among instances that are permitted to interact, while still enforcing isolation policies. This allows eGPU to exploit the full NVLink bandwidth across containers without compromising resource boundaries.

Additionally, eGPU enables efficient intra-GPU communication when multiple instances are co-located on the same GPU. eGPU detects such co-location through GPU device

identifiers and supports two complementary communication mechanisms: (1) shared host memory coordination, where ranks perform data exchange by reading from and writing to a shared memory region on the host via PCIe; and (2) CUDA Inter-Process Communication (IPC), where ranks notify each other of send/receive buffer addresses in device memory and perform direct device-to-device (D2D) memory copies.

Together, these mechanisms enable eGPU to support both inter-GPU and intra-GPU communication efficiently in shared environments, while maintaining fault isolation and resource guarantees. At the same time, eGPU's resource-limiting capabilities ensure that each user job only accesses its designated share of GPU resources.

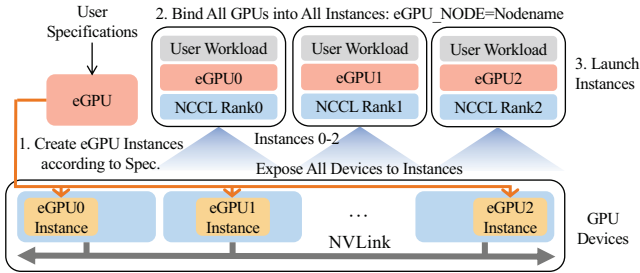


Fig. 7: eGPU NVLink Support for Efficient Training.

#### E. Scalable Instance Control with K8s

To guarantee scalability, we introduce a real-time, precise GPU resource control system that enhances K8s with capabilities to directly manage eGPU instances. This solution consists of three main components and two communication links among them: ① *eGPU Device Plugin*, ② *eGPU Container Toolkit*, and ③ *eGPU Runtime Daemon*, as shown in Figure 8.

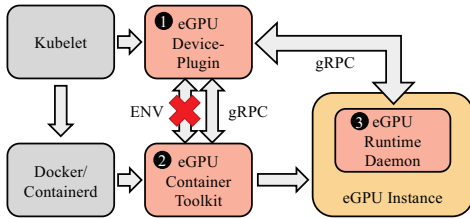


Fig. 8: eGPU Scalable Instance Control with K8s.

Firstly, the *eGPU Device Plugin* is introduced to inform K8s about partitionable GPUs, enabling the cluster scheduler to make informed scheduling decisions. To ensure eGPU is transparent to user tasks, we design the *eGPU Container Toolkit* with an instance control interface compliant with OCI (Open Container Initiative) standards [10]. This allows eGPU to integrate seamlessly with various mainstream container technologies such as Singularity [29], Docker [19], Containerd [2], Pouch [13], and Podman [12]. To support runtime adjustments, which differ from the native K8s static scheme, we introduce the *eGPU Runtime Daemon* within the instances.

To avoid resource misuse, we introduce the first link between *eGPU Device Plugin* and *eGPU Instance Toolkit*.

We have abolished the mechanism of writing environment variables to start the instance. Instead, we have replaced it with gRPC [4] communication for configuration, such as mounting and retrieving device information. To enable real-time, elastic resource adjustments during task execution, we established the second communication link between *eGPU Device Plugin* and *eGPU Runtime Daemon*. When a running instance necessitates dynamic adjustments, *eGPU Device Plugin* will communicate with *eGPU Runtime Daemon* inside the instance, facilitating flexible resource management beyond the static capabilities of native K8s.

## V. IMPLEMENTATION

### A. Instance Startup

eGPU is implemented with around 100,000 lines of code mainly in C and Golang. In order to implement CUDA API interception, the CUDA library in the instance needs to be replaced first. To be transparent to upper users, when an instance starts, besides building the original GPU instance environment, we use a self-developed eGPU instance runtime that will also bring the replacement library and other files required by the eGPU, so that no pre-modification is required for user tasks and automatic configuration is achieved.

### B. CUDA API Interception Workflow

In eGPU, we have an *eGPU Wrapper* that is responsible for API interception. The CUDA API interception workflow is illustrated in Figure 9. Every time a user program calls a CUDA API, it will first go through *eGPU Wrapper* that will perform corresponding processing according to the API type before calling the real CUDA API. When necessary, *eGPU Wrapper* will trigger the check action and communicate with *eGPU Runtime Daemon*, where all GPU-related query work is completed. *eGPU Wrapper* will determine whether to execute the related API according to the returned query results. The available GPU memory and computing power can be set through the environment variables in the instance.

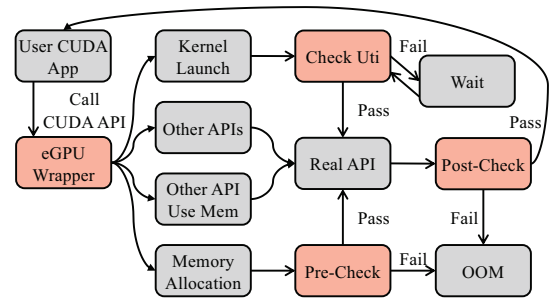


Fig. 9: CUDA API Interception Workflow.

When the user program executes the GPU memory allocation related APIs, eGPU will perform a pre-check to determine whether the current instance has enough remaining GPU memory to satisfy the allocation operation. If not, the API will directly return failure and no real memory allocation will occur. When the user program executes other APIs that

may use GPU memory, eGPU will perform a post-check after executing the API to determine whether the GPU memory used by the current instance has exceeded the set value. If not, the API directly returns success, otherwise, it returns failure. When the user program executes kernel launch related APIs, eGPU will check whether the GPU compute utilization of the current instance exceeds the set value. If it does not exceed the set value, it will be executed directly. Otherwise, it will wait for a period of time and then check again. In this way, we realize the partition of computing resources. For all other APIs, there is no need to check, we will just call the real CUDA API directly.

## VI. EVALUATION

Through our evaluation, we aim to answer three questions:

- ① Can eGPU provide elasticity in GPU utilization and workload performance with FAA?
- ② Can eGPU improve efficiency to better guarantee user SLAs by overcoming communication bottlenecks?
- ③ Can eGPU ensure fault isolation, and is it scalable to be deployed in production-scale data center clusters?

### A. Setup

**Platform:** To show eGPU's support of heterogeneous GPUs, the evaluations are carried out on platforms featuring three distinct types of GPUs. The first platform comprises eight NVIDIA A100 40 GB PCIe GPUs, Intel Xeon Platinum 8269CY CPUs, and 768 GB of host memory. The second platform boasts an Intel Xeon Platinum 8260 CPU, eight NVIDIA Tesla V100-SXM2-32GB GPUs, and 1.5 TB of host memory. The third platform is equipped with Intel Xeon Platinum 8260 CPUs, eight NVIDIA T4 15 GB GPUs, and 512G of host memory. Additionally, we gather data from one of our production clusters with heterogeneous GPUs.

**Baseline:** We include four academic-provided priority-based solutions: REEF [22] and Orion [48] as spatial sharing solutions, while Antman [58] and TGS [56] as temporal sharing solutions. Additionally, we select two industry-provided partition-based solutions MPS [8] and MIG [7], which are also spatial-sharing solutions.

**Workloads:** We evaluate eGPU using a range of representative workloads across computer vision and natural language processing domains. For vision tasks, we include two ResNet [23] variants ResNet-50 and ResNet-101, as well as two private models provided by our users. For NLP, we use BERT [18], Qwen [14], and LLaMA [51].

**Methodology:** Our evaluations aim to test if eGPU can achieve our design goals from those three aspects: elasticity, efficiency, and scalability. To show eGPU's ability to support heterogeneous GPUs, for each experiment, we present results from one of the above three platforms. The conclusion among heterogeneous GPUs will be the same. The first platform is used for MIG and LLM related experiments, the second platform is used for experiments of adjusting instances during runtime, and the third platform is used for the rest.

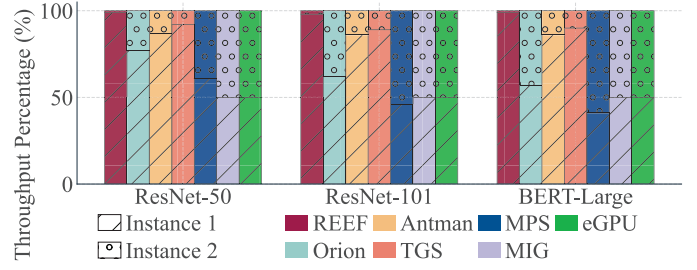


Fig. 10: Precise Partitioning of Instances.

### B. Providing Elasticity in Partitioning

1) *Elastic Partitioning in Instance Size:* Firstly, we evaluate eGPU's partitioning precision on instance sizes. We share one GPU with two independent instances and we compare their training throughput ratios. We use batch sizes of 32 for ResNet-50 and ResNet-101, and a batch size of 8 for BERT-Large (same for later experiments if not stated otherwise). For illustration purposes, we set 50%/50% for partition-based designs, including eGPU and MPS, and two 3/7 instances for MIG. Figure 10 presents the results across the different sharing solutions. Attributed to its robust performance isolation, MIG demonstrates a comparable precision as eGPU with throughput ratios close to an equitable 50%/50% split. In contrast, MPS shows a slight deviation from the 50%/50% partitioning due to interference. Meanwhile, we also show results from other priority-based solutions for illustration purposes. Note that they are designed to prioritize one instance over the other. So they clearly show leading resource dominance by the higher-priority instance.

2) *Elastic Adjustment during Runtime:* Secondly, we explore the system's ability to adjust instance sizes precisely and elastically during runtime, as designed in Section IV-C. We test training ResNet-50 with a batch size of 16 under two cases: 1. Adjust GPU resources allocated for one instance. 2. Adjust GPU resources allocated for two concurrent training instances independently. For both experiments, we adjust the resource percentage by 10% at a time. The resulting changes in training throughput are shown in Figure 11a and Figure 11b, respectively. For better illustration, we also include a result for native GPU to show overheads for sharing. From the results, eGPU can precisely and elastically adjust GPU resources allocated to instances.

3) *Benefits of FAA:* Lastly, to investigate the impact of our designed sharing method FAA, we use different instance sizes with ResNet-50 workload. We measure its impact by improved training throughput and reduced inference response time (RT). As shown in Figure 12a, the default coarse time slicing (round robin among instances) incurs training throughput from 56 up to 192 images per second (ips) under different instance sizes. By using FAA, the throughput is improved to a range from 63 to 192 ips. Meanwhile, from Figure 12b inference overhead drops by 10.1% on average (24.1% at most) by using FAA.

TABLE II: Low Overhead in Throughput for ML Workloads.

|            | ResNet-50 |         |         | BERT   |        |        | LLaMA |       |       | Private User Model 1 |          |          | Private User Model 2 |          |          |
|------------|-----------|---------|---------|--------|--------|--------|-------|-------|-------|----------------------|----------|----------|----------------------|----------|----------|
| Throughput | bs=4      | bs=16   | bs=64   | bs=4   | bs=16  | bs=64  | bs=4  | bs=16 | bs=64 | bs=4                 | bs=16    | bs=64    | bs=4                 | bs=16    | bs=64    |
| Native     | 424.607   | 608.036 | 664.216 | 41.608 | 44.880 | 45.917 | 0.852 | 2.610 | 5.910 | 756.659              | 1203.132 | 1339.478 | 1048.248             | 2818.940 | 4792.150 |
| eGPU       | 420.032   | 608.159 | 642.188 | 41.266 | 44.762 | 45.868 | 0.834 | 2.592 | 5.893 | 793.304              | 1210.332 | 1346.200 | 1102.955             | 2900.120 | 4800.020 |

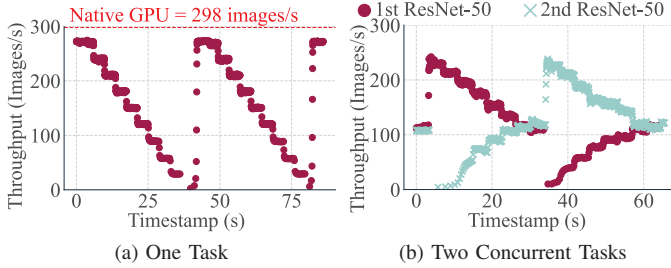


Fig. 11: Adjustment of Instance Sizes during Runtime.

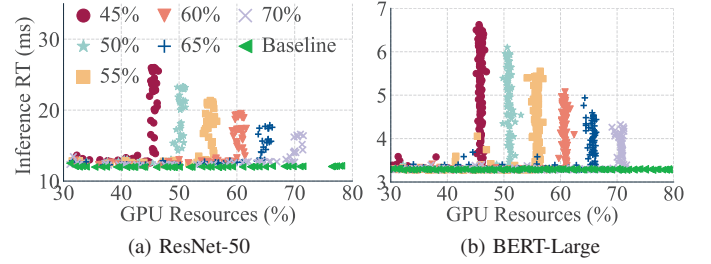


Fig. 13: Guarantee Response Time for Inference Tasks.

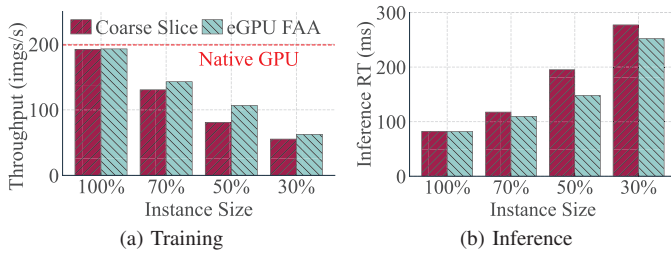


Fig. 12: Comparison between Coarse Slicing and FAA.

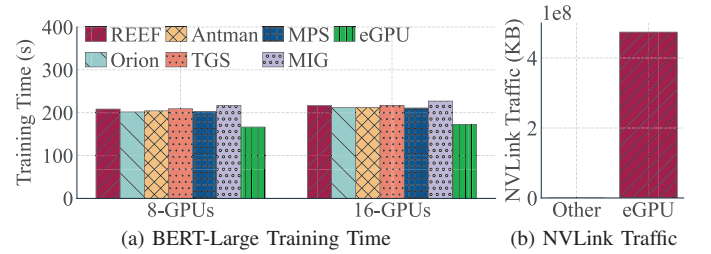


Fig. 14: Improved Training Time due to NVLink.

### C. Improving Efficiency for User Workloads

1) *Low Overhead for ML Workloads:* Firstly, we evaluate eGPU's overhead for ML workloads, indicated by throughput (queries per second). Table II presents the results of inference throughput of various deep learning models (ResNet50, BERT, LLaMA, and two private models from users) with different batch sizes from 4 to 64 using eGPU and native GPU. To highlight eGPU's overhead clearly, we compare it directly against native GPU performance under a full 100% instance allocation. The results show that the eGPU performs comparably to the native GPU for all batch sizes and all models. Even in the worst case, it still can achieve 96.7% of the original throughput. Altogether, there is minimal performance loss in general applications with eGPU, and it is less than 4% even when the batch size is large.

2) *Guarantee RT for Inference Tasks:* In this experiment, we evaluate whether eGPU can guarantee SLAs for inference tasks using ResNet-50 and BERT-Large as examples. Instances are set with varying user-requested GPU resource limiting levels as examples (45% to 70%), and we report the inference RT under different actual GPU resource usages for applications. The baseline configuration has no GPU resource limitations. The results, presented in Figure 13, reveal two key observations. Firstly, we observe that eGPU does not significantly impact inference RT. Before the actual GPU resources reach

the imposed limitation, the RT remains stable across different resource levels, thus ensuring SLAs for inference. Secondly, we note that GPU resources can be effectively controlled by adjusting the limiting level. This is evidenced by the rapid increase in inference RT once the actual GPU resource usage reaches its requested limit. This finding demonstrates that our approach is effective in managing resource allocation to instances, thereby preventing any significant impact on overall system performance. To the best of our knowledge, among existing temporal sharing solutions, no previous work has achieved such precise resource limitation with minimal impact on inference RT.

3) *Reduce Execution Time for Training Tasks:* We next investigate the benefits brought by leveraging the high-performance NVLink as designed in Section IV-D. We compare eGPU with other solutions for distributed training BERT-Large with K8s in terms of end-to-end training time. Note that TGS is designed to use Docker, which inherently prevents it from utilizing NVLink regardless of the configuration. Other solutions in this experiment are implemented in K8s to ensure a fair comparison under standard container-based isolation. If fault isolation is not required, these solutions will be able to use NVLink. We are training 400 steps with a batch size of 128. The experimental results shown in Figure 14a indicate that the proposed eGPU can achieve better performance for

the BERT-Large model. By overcoming communication bottlenecks, eGPU achieves a 1.21x to 1.31x speedup in training efficiency. The monitored NVLink traffic during the training is shown in Figure 14b.

4) *Case Study 1: LLM Workloads*: LLMs have emerged as a highly popular topic in recent years. Although LLMs typically demand substantial resources, making them challenging candidates for GPU sharing, we conducted a case study to explore whether eGPU can effectively handle such demands if required. We evaluate the performance of sharing a LLaMA-7B inference task with a maximum sequence length of 256 alongside ResNet-50 training with a batch size of 8. By adjusting the resource allocation percentages for LLaMA, we measure LLaMA’s inference RT and ResNet-50’s training throughput. Figure 15 presents the performance results under different resource allocation scenarios. The bars in the figure represent LLaMA inference RT. When dedicated, the RT is 5.7 seconds. Without sharing, as the allocated resources for LLaMA decrease from 70% to 30%, its RT increases from 8.1 seconds to 19.2 seconds, indicating turning down resources for LLM will significantly hurt its performance. When resources are shared, the RT rises from 8.2 seconds to 20.0 seconds with a similar trend. Meanwhile, the line in the figure indicates the ResNet-50 training throughput during these sharing scenarios.

5) *Case Study 2: Priority-based Situation.*: We show a real-world case study provided by a user who utilized eGPU in a priority-based scenario. In this case, the high-priority task was an inference job focused on image classification, while the low-priority task involved DNN training. The user’s objective was to maintain overall GPU utilization at approximately 80%, while ensuring real-time performance for the inference task and allocating remaining resources to the training job. As shown in the monitored trace in Figure 16, the user initially started the inference job with a full partition at timestamp 0, observing a GPU utilization of approximately 11%. When the training job arrived at timestamp 48, the user adjusted the partition sizes, allocating 69% of the GPU to the training job and reducing the inference job’s partition to 31%. This adjustment successfully raised overall GPU utilization to around 80%, while ensuring the inference job’s response time remained stable, ranging between 20 ms and 24 ms.

#### D. Ensuring Scalability to Production Scale

1) *Fault Isolation*: In this experiment, we evaluate the advantages of eGPU’s fault isolation capabilities. We run two concurrent ResNet-50 training tasks, each with a batch size of 16, under three different configurations: MPS with a 50%/50% resource split to represent solutions that manage kernels, MIG with a 3/7+4/7 instance split to represent static sharing, and eGPU. During the experiment, one of the training tasks will eventually encounter a fatal error. We compare the training throughput and GPU utilization across these configurations. Figure 17 reports the performance metrics before and after the fatal error occurs. Initially, all configurations achieve comparable training throughput and GPU utilization. However,

the results diverge significantly after one instance is stopped due to the fatal error.

For MPS, although it achieves the highest throughput and GPU utilization before the error, the entire GPU crashes when the error occurs in one instance. In the case of MIG, terminating one instance leads to a substantial drop in GPU utilization from 91% to 39%. In contrast, eGPU maintains a high GPU utilization of 85% under the same conditions. This is because MIG provides strong physical isolation, ensuring that even if a task temporarily does not require computing resources, the resources allocated to its MIG instance cannot be used by other tasks. On the other hand, eGPU, when only limiting GPU memory, allows other tasks to utilize the remaining computing resources when one task is terminated, thereby maximizing the overall GPU utilization.

2) *Higher Instance Count*: To evaluate eGPU’s ability to maintain stable sharing under high instance counts, we conduct an experiment on a single NVIDIA H20 GPU by concurrently launching 10 inference tasks, consisting of five Llama-3.2-1B-Instruct models and five Qwen2.5-0.5B-Instruct models. Because the Llama model is larger, each Llama instance is assigned 13% of the GPU memory, while each Qwen instance is assigned 7%, totaling 100% full-GPU allocation. These tasks represent a heterogeneous configuration and a high instance count.

TABLE III: Inference Throughput for 10 Concurrent Instances

| Timestamp (s) | 10    | 20    | 30    | 40    | 50    | 60    | 70    | 80    | 90    | 100   |
|---------------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|
| 1-st Llama    | 23.29 | 23.29 | 23.28 | 23.29 | 23.28 | 23.28 | 23.28 | 23.28 | 23.27 | 23.27 |
| 2-nd Llama    | 23.28 | 23.29 | 23.27 | 23.28 | 23.28 | 23.29 | 23.28 | 23.28 | 23.28 | 23.27 |
| 3-rd Llama    | 23.29 | 23.29 | 23.28 | 23.28 | 23.28 | 23.28 | 23.28 | 23.28 | 23.28 | 23.27 |
| 4-th Llama    | 23.28 | 23.29 | 23.28 | 23.27 | 23.29 | 23.28 | 23.28 | 23.29 | 23.28 | 23.27 |
| 5-th Llama    | 23.29 | 23.28 | 23.28 | 23.28 | 23.29 | 23.29 | 23.28 | 23.28 | 23.28 | 23.27 |
| 1-st Qwen     | 25.56 | 25.54 | 25.52 | 25.54 | 25.54 | 25.52 | 25.54 | 25.53 | 25.53 | 25.53 |
| 2-nd Qwen     | 25.57 | 25.53 | 25.53 | 25.53 | 25.53 | 25.53 | 25.54 | 25.54 | 25.53 | 25.53 |
| 3-rd Qwen     | 25.54 | 25.54 | 25.53 | 25.54 | 25.53 | 25.53 | 25.53 | 25.53 | 25.55 | 25.52 |
| 4-th Qwen     | 25.54 | 25.53 | 25.53 | 25.54 | 25.55 | 25.53 | 25.53 | 25.53 | 25.54 | 25.52 |
| 5-th Qwen     | 25.56 | 25.53 | 25.55 | 25.54 | 25.53 | 25.55 | 25.54 | 25.52 | 25.52 | 25.53 |

During execution, each instance continuously reports its inference throughput (tokens/s). As Table III shows, across all five Llama tasks, query-per-second (QPS) remains within a narrow band of 23.27–23.29 tokens/s, showing a variation of less than 0.1% across time and across replicas. The five Qwen tasks exhibit similarly stable behavior, with QPS ranging from 25.52–25.57 tokens/s, corresponding to a fluctuation within 0.2%. From these results, we can see that eGPU naturally supports partitioning into a high number of instances with our partition-based FAA design.

3) *Saving GPU Resources with eGPU*: In addition, we disclose some censored data on how eGPU helps save GPU resources for our internal business. Table IV shows the number of GPUs required to perform certain ML tasks with and without eGPU. In the traditional way, each workload will require a whole GPU no matter how much resource it actually requires. With eGPU, multiple instances can coexist on the same GPU, leading to saved resources. As can be seen from the table, eGPU is able to achieve significant GPU resource

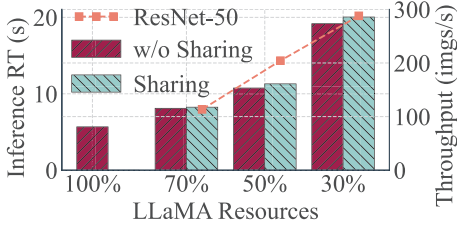


Fig. 15: Sharing with LLM Workloads.

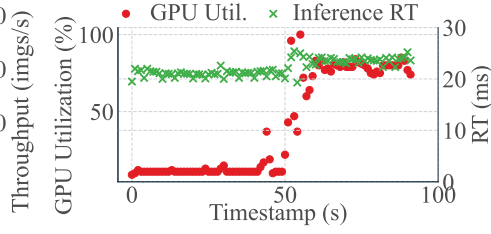


Fig. 16: Priority-based Sharing.

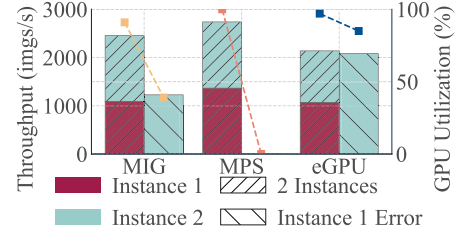


Fig. 17: Fault Isolation in eGPU.

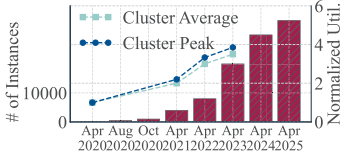


Fig. 18: Active Instances and Improved GPU Utilization.

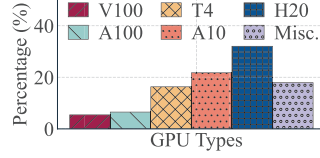


Fig. 19: Deployment on Heterogeneous GPUs.

savings for each workload. The ratios presented in the table show that our system can achieve up to  $8\times$  GPU savings for visual feature processing,  $7.9\times$  GPU savings for text generation,  $2\times$  GPU savings for video tag generation, and  $3\times$  GPU savings for content analyzing.

TABLE IV: Saving GPU Resources

| Workloads                 | Native | w/ eGPU | Ratio       |
|---------------------------|--------|---------|-------------|
| Visual Feature Processing | 2,000  | 250     | $8\times$   |
| Text Generation           | 1,500  | 190     | $7.9\times$ |
| Video Tag Generation      | 100    | 50      | $2\times$   |
| Content Analyzing         | 90     | 30      | $3\times$   |

4) *Instances Deployed and Improved Cluster Usage:* To illustrate the cluster GPU utilization improvement provided by eGPU, we collect statistics since its deployment in April 2020. The utilization right before April 2020, with no GPU-sharing, serves as the baseline. Note that as the jobs over time are different, we use system metrics as side indicators, as the data is from the same production cluster. The bars in Figure 18 show the number of active eGPU instances over the past four years. The number of instances increased from 10 in April 2020 to over 10,000 by April 2023 and is still growing, indicating strong demand for GPU sharing.

At the same time, the lines in Figure 18 show the normalized cluster GPU utilization since eGPU deployment. We also observe a significant increase in GPU utilization from April 2020 to April 2023, with a ratio of more than  $3\times$  for both cluster average and peak. As more and more eGPU instances are deployed, the GPU utilization grows over time.

5) *Number of Heterogeneous GPUs Deployed:* In terms of deployment with heterogeneous GPUs, Figure 19 shows the percentage of each GPU type deployed with eGPU in one cluster. As shown, the most commonly deployed GPU type is the H20 with more than 30%, followed by the A10

with a share of around 22%, and miscellaneous types with around 18%. Additionally, there are around 16% of T4, 7% of A100, and 5% of V100. Taking all clusters into account, eGPU has been deployed to far more than 10,000 GPUs in total. These statistical numbers suggest that the eGPU system is a scalable and generic solution that is able to effectively support heterogeneous GPU types and accommodate diverse production needs.

## VII. LESSONS LEARNED

### Lesson 1: Adapting to Dynamic Workload Demands.

Like prior works [25], [55], we also observe that the demand for GPU resources tends to follow a ‘tide-style’ pattern in production clusters, with iterative fluctuations over time. This variability underscores the critical importance of elastic resource allocation, where GPUs can dynamically adjust their resource allocation in real-time to match workload demands. This capability is essential for optimizing overall cluster efficiency, especially in large-scale deployments.

Initially, we explored the use of MIG [7] to address these needs, but several limitations became apparent. The requirement to reset the GPU to toggle MIG on or off created substantial disruptions in our dynamic environments. Additionally, MIG’s fixed partition sizes and lack of NVLink support led to performance degradation in our workloads. While MIG showed promise in highly stable and task-heavy scenarios, such conditions were rare in our production clusters, limiting its practical applicability.

One of our most important insights relates to the nature of the GPU kernels in our production ML workloads, which tend to have very short execution times, often less than 0.1ms. Coupled with the iterative nature of these tasks, we realized that traditional approaches, focused on managing the duration of individual kernels, were not well-suited to our needs. Instead, the challenge was in managing the large volume of recurring, short-duration kernels. This insight led us to develop the finer-grained time-slicing design, which proved far more effective in improving resource utilization and job performance in production.

### Lesson 2: Balancing Efficiency and Responsiveness.

As eGPU evolved alongside the introduction of more advanced GPU models like the A100, we encountered unexpected efficiency challenges. These newer, high-performance GPUs processed kernels far more quickly than earlier generations, creating a paradox where eGPU became the bottleneck

due to the CPU's slower processing capabilities. This highlighted a critical lesson: without optimizations, even powerful hardware can underperform if supporting systems lag behind.

To address this, we increased eGPU's parallelism by introducing additional threads, allowing the CPU-based processes of eGPU to better keep pace with the advanced GPUs. This adjustment alleviated the bottleneck and enhanced overall system efficiency, enabling us to fully leverage the performance potential of cutting-edge GPUs.

Another key lesson involved the importance of maintaining user SLAs in production environments. While our initial focus with eGPU was on precise instance partitioning and size control, we quickly realized that pursuing partition accuracy alone is insufficient if it compromises user SLAs. If the user SLAs are violated, then the partition accuracy is meaningless no matter how high it is. This taught us that besides partition accuracy, user job performance is also crucial. Balancing these priorities is essential for achieving both technical precision and operational success.

### Lesson 3: Supporting Heterogeneous Environments.

A key lesson from our scalability journey was the need to support heterogeneous GPU environments. As clusters are built and expanded over time, they inevitably include a mix of different GPU types. This diversity requires a system that can effectively manage and optimize across various architectures, ensuring that performance remains consistent even in a mixed hardware landscape. Developing eGPU to be compatible and adaptable across multiple GPU types was critical in maintaining scalability and flexibility.

We also learned the importance of compatibility in scaling eGPU. Initially, we designed eGPU to integrate with platform-provided containers, which, while functional, required additional modifications to enable full eGPU capabilities. This approach, however, increased container startup times and created a tight dependency on specific startup scripts, limiting flexibility when extending to other platforms. To overcome this limitation, we introduced the *eGPU Container Toolkit*, which allows eGPU functionality to be injected into containers during startup. This toolkit removes the need to modify container images directly, streamlining the deployment process and improving scalability.

## VIII. DISCUSSION

### A. Isolation Concerns

As highlighted earlier, the primary objective of eGPU is to optimize resource utilization while maintaining job performance, with security-related considerations being a secondary consideration. Nonetheless, our current methods of resource sharing and isolation within eGPU have proven to be quite effective. Regarding the GPU resources in containers, eGPU employs CUDA contexts to manage resources and data on the GPU. These contexts act similarly to CPU processes, offering a high degree of isolation. This design means that it is not feasible for processes in different containers to access each other's GPU resources or data. Such a setup ensures that the instances running on the same GPU are adequately isolated,

akin to the isolation standards observed in CPU processes. This confidence is further bolstered by many public reports regarding the widespread adoption and validation of container-based isolation across various industries [1], [3], affirming its reliability and security.

### B. Future Directions

We envision extending eGPU to support spatial sharing as a promising direction. eGPU currently focuses on temporal sharing, a design informed by extensive study of production clusters and our operational experience. While spatial sharing solutions such as MPS and MIG have proven highly useful, they still face notable limitations in practice: MPS lacks strong fault isolation, while MIG offers isolation yet sacrifices elasticity and suffers from rigid resource boundaries.

Importantly, eGPU operates orthogonally to existing spatial mechanisms. In environments where job correctness is trusted, eGPU can complement MPS to provide flexible temporal multiplexing. Conversely, when isolation is prioritized over peak efficiency, eGPU can operate alongside MIG to retain isolation guarantees while improving resource utilization.

Looking ahead, we see a valuable opportunity to unify these paradigms. Specifically, designing a future system that combines elastic spatial-temporal sharing with strong isolation (e.g., eGPU + enhanced MIG), efficient communication, and production-grade fault tolerance would meaningfully benefit large-scale deployments. We believe that such an integration represents a compelling next step for production GPU sharing systems.

## IX. CONCLUSION

The advancement of high-performance GPUs has led to many workloads underutilizing available GPU capacity, creating a growing need for effective GPU-sharing. In this paper, we highlight the unique challenges we encountered and how we addressed them with GPU sharing in production environments. We then present our design of eGPU, a production-ready GPU-sharing solution that is elastic, efficient, and scalable. eGPU introduces a fine-grained anti-starving algorithm for elastic GPU resource partitioning and adjustments. Efficiency is enhanced through high-speed communication among instances with full NVLink and NCCL support. eGPU is scalable by optimizing K8s and has been deployed in production-scale clusters with over 10,000 GPUs for five years. Our evaluation shows that eGPU can improve job efficiency by 21% to 31% than SOTA sharing solutions, save up to 8× the number of GPUs required, and improve cluster GPU utilization by more than 3×.

## X. ACKNOWLEDGEMENT

We thank the HPCA anonymous reviewers for their valuable feedback.

## REFERENCES

- [1] "A Krispr Approach to Kubernetes Infrastructure," <https://medium.com/airbnb-engineering/a-krispr-approach-to-kubernetes-infrastructure-a0741cff4e0c>.
- [2] "Containerd," <https://containerd.io/>.
- [3] "Google Kubernetes Engine," <https://cloud.google.com/kubernetes-engine?hl=en>.
- [4] "gRPC," <https://grpc.io/>.
- [5] "MTIA: META'S First Generation of AI Accelerators," <https://atscaleconference.com/videos/mtia-metas-first-generation-of-ai-accelerators/>.
- [6] "NVIDIA Blackwell Architecture Technical Brief," <https://resources.nvidia.com/en-us-blackwell-architecture>.
- [7] "NVIDIA Multi-Instance GPU User Guide," [https://docs.nvidia.com/datacenter/tesla/pdf/NVIDIA\\_MIG\\_User\\_Guide.pdf](https://docs.nvidia.com/datacenter/tesla/pdf/NVIDIA_MIG_User_Guide.pdf).
- [8] "NVIDIA Multi-Process Service," [https://docs.nvidia.com/deploy/pdf/CUDA\\_Multi\\_Process\\_Service\\_Overview.pdf](https://docs.nvidia.com/deploy/pdf/CUDA_Multi_Process_Service_Overview.pdf).
- [9] "NVLink," <https://www.nvidia.com/en-us/design-visualization/nvlink-bridges/>.
- [10] "OCI Standards," <https://opencontainers.org/>.
- [11] "PCIe Gen6," <https://pcisig.com/pci-express-6.0-specification>.
- [12] "Podman," <https://podman.io/>.
- [13] "PouchContainer," <https://pouchcontainer.io/>.
- [14] J. Bai, S. Bai, Y. Chu, Z. Cui, K. Dang, X. Deng, Y. Fan, W. Ge, Y. Han, F. Huang *et al.*, "Qwen Technical Report," *arXiv preprint arXiv:2309.16609*, 2023.
- [15] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, "Borg, Omega, and Kubernetes," *Communications of the ACM*, vol. 59, no. 5, pp. 50–57, 2016.
- [16] S. Choi, S. Lee, Y. Kim, J. Park, Y. Kwon, and J. Huh, "Serving Heterogeneous Machine Learning Models on Multi-GPU Servers with Spatio-Temporal Sharing," in *2022 USENIX Annual Technical Conference (USENIX ATC 22)*, 2022, pp. 199–216.
- [17] S. Choi, S. Lee, Y. Kim, J. Park, Y. Kwon, and J. Huh, "Serving Heterogeneous Machine Learning Models on Multi-GPU Servers with Spatio-Temporal Sharing," in *2022 USENIX Annual Technical Conference (USENIX ATC 22)*, 2022, pp. 199–216.
- [18] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of Deep Bidirectional Transformers for Language Understanding," *arXiv preprint arXiv:1810.04805*, 2018.
- [19] Docker, <https://www.docker.com/>.
- [20] M. Dowty and J. Sugerman, "GPU Virtualization on VMware's Hosted I/O Architecture," *ACM SIGOPS Operating Systems Review*, vol. 43, no. 3, pp. 73–82, 2009.
- [21] A. Gujarati, R. Karimi, S. Alzayat, W. Hao, A. Kaufmann, Y. Vigfusson, and J. Mace, "Serving DNNs like Clockwork: Performance Predictability from the Bottom Up," in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, 2020, pp. 443–462.
- [22] M. Han, H. Zhang, R. Chen, and H. Chen, "Microsecond-scale Preemption for Concurrent GPU-accelerated DNN Inferences," in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 2022, pp. 539–558.
- [23] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
- [24] C.-H. Hong, I. Spence, and D. S. Nikolopoulos, "GPU Virtualization and Scheduling Methods: A Comprehensive Survey," *ACM Computing Surveys (CSUR)*, vol. 50, no. 3, pp. 1–37, 2017.
- [25] Q. Hu, P. Sun, S. Yan, Y. Wen, and T. Zhang, "Characterization and Prediction of Deep Learning Workloads in Large-scale GPU Datacenters," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC '21. New York, NY, USA: Association for Computing Machinery, 2021. [Online]. Available: <https://doi.org/10.1145/3458817.3476223>
- [26] M. Jeon, S. Venkataraman, A. Phanishayee, J. Qian, W. Xiao, and F. Yang, "Analysis of Large-Scale Multi-Tenant GPU Clusters for DNN Training Workloads," in *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, 2019, pp. 947–960.
- [27] M. Jeon, S. Venkataraman, A. Phanishayee, J. Qian, W. Xiao, and F. Yang, "Analysis of Large-Scale Multi-Tenant GPU Clusters for DNN Training Workloads," in *2019 USENIX Annual Technical Conference (USENIX ATC 19)*. Renton, WA: USENIX Association, Jul. 2019, pp. 947–960. [Online]. Available: <https://www.usenix.org/conference/atc19/presentation/jeon>
- [28] W. Jia, Z. Cao, L. Wang, J. Fu, X. Chi, W. Gao, and L.-W. Wang, "The Analysis of a Plane Wave Pseudopotential Density Functional Theory Code on a GPU Machine," *Computer Physics Communications*, vol. 184, no. 1, pp. 9–18, 2013.
- [29] G. M. Kurtzer, V. Sochat, and M. W. Bauer, "Singularity: Scientific Containers for Mobility of Compute," *PLOS ONE*, vol. 12, pp. 1–20, 05 2017.
- [30] M. Li, W. Xiao, H. Yang, B. Sun, H. Zhao, S. Ren, Z. Luan, X. Jia, Y. Liu, Y. Li *et al.*, "EasyScale: Elastic Training with Consistent Accuracy and Improved Utilization on GPUs," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2023, pp. 1–14.
- [31] P. Li, Y. Luo, N. Zhang, and Y. Cao, "Heterospark: A Heterogeneous CPU/GPU Spark Platform for Machine Learning Algorithms," in *2015 IEEE International Conference on Networking, Architecture and Storage (NAS)*. IEEE, 2015, pp. 347–348.
- [32] Y. Lin, S. Han, H. Mao, Y. Wang, and B. Dally, "Deep Gradient Compression: Reducing the Communication Bandwidth for Distributed Training," in *International Conference on Learning Representations*, 2018. [Online]. Available: <https://openreview.net/forum?id=SkhQHMW0W>
- [33] Y. Mao, Y. Fu, S. Gu, S. Vhaduri, L. Cheng, and Q. Liu, "Resource Management Schemes for Cloud-native Platforms with Computing Containers of Docker and Kubernetes," *arXiv preprint arXiv:2010.10350*, 2020.
- [34] J. Nickolls and W. J. Dally, "The GPU Computing Era," *IEEE micro*, vol. 30, no. 2, pp. 56–69, 2010.
- [35] NVIDIA, "NCCL Communicators," <https://docs.nvidia.com/deeplearning/ncll/user-guide/docs/usage/communicators.html>.
- [36] NVIDIA, "NVIDIA Collective Communication Library (NCCL)," <https://developer.nvidia.com/nccl>.
- [37] NVIDIA, "NVIDIA A100 TENSOR CORE GPU," <https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/a100/pdf/nvidia-a100-datasheet-nvidia-us-2188504-web.pdf>, 2023.
- [38] NVIDIA, "NVIDIA H100 TENSOR CORE GPU," <https://resources.nvidia.com/en-us-tensor-core/nvidia-tensor-core-gpu-datasheet>, 2023.
- [39] NVIDIA CONTAINERS AND DEEP LEARNING FRAMEWORKS, <http://docs.nvidia.com/deeplearning/dgx/pdf/User-Guide.pdf>.
- [40] NVIDIA GPU Cloud, <https://www.nvidia.com/en-us/gpu-cloud/>.
- [41] M. Pandey, M. Fernandez, F. Gentile, O. Isayev, A. Tropsha, A. C. Stern, and A. Cherkasov, "The Transformational Role of GPU Computing and Deep Learning in Drug Discovery," *Nature Machine Intelligence*, vol. 4, no. 3, pp. 211–221, 2022.
- [42] C. Reano and F. Silla, "InfiniBand Verbs Optimizations for Remote GPU Virtualization," in *2015 IEEE International Conference on Cluster Computing*, 2015.
- [43] C. Reano and F. Silla, "A Comparative Performance Analysis of Remote GPU Virtualization Over Three Generations of GPUs," in *2017 46th International Conference on Parallel Processing Workshops (ICPPW)*. IEEE, 2017, pp. 121–128.
- [44] C. Root and T. Mostak, "MapD: A GPU-powered Big Data Analytics and Visualization Platform," in *ACM SIGGRAPH 2016 Talks*, 2016, pp. 1–2.
- [45] H. Shen, L. Chen, Y. Jin, L. Zhao, B. Kong, M. Philipose, A. Krishnamurthy, and R. Sundaram, "Nexus: a GPU Cluster Engine for Accelerating DNN-based Video Analysis," ser. SOSP '19. New York, NY, USA: Association for Computing Machinery, 2019, p. 322–337. [Online]. Available: <https://doi.org/10.1145/3341301.3359658>
- [46] L. Shi, H. Chen, and J. Sun, "vCUDA: GPU accelerated high performance computing in virtual machines," in *2009 IEEE International Symposium on Parallel Distributed Processing*, May 2009, pp. 1–11.
- [47] D. Steinkraus, I. Buck, and P. Y. Simard, "Using GPUs for Machine Learning Algorithms," in *Eighth International Conference on Document Analysis and Recognition (ICDAR'05)*. IEEE, 2005, pp. 1115–1120.
- [48] F. Strati, X. Ma, and A. Klimovic, "Orion: Interference-aware, Fine-grained GPU Sharing for ML Applications," in *Proceedings of the Nineteenth European Conference on Computer Systems*, 2024, pp. 1075–1092.
- [49] Y. Suzuki, S. Kato, H. Yamada, and K. Kono, "Gpuvmm: Why not virtualizing gpus at the hypervisor?" in *USENIX Annual Technical Conference*, 2014, pp. 109–120.
- [50] H. Tang, S. Gan, C. Zhang, T. Zhang, and J. Liu, "Communication Compression for Decentralized Training," in *Proceedings of the 32nd International Conference on Neural Information Processing Systems*,

- ser. NIPS'18. Red Hook, NY, USA: Curran Associates Inc., 2018, p. 7663–7673.
- [51] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, “LLaMA: Open and Efficient Foundation Language Models,” 2023.
  - [52] R. Vaño, I. Lacalle, P. Sowiński, R. S-Julián, and C. E. Palau, “Cloud-native Workload Orchestration at the Edge: A Deployment Review and Future Directions,” *Sensors*, vol. 23, no. 4, p. 2215, 2023.
  - [53] J. Wang, B. Yuan, L. Rimanic, Y. He, T. Dao, B. Chen, C. Re, and C. Zhang, “Fine-tuning Language Models over Slow Networks using Activation Compression with Guarantees,” 2023.
  - [54] Y. Wang, B. Li, R. Luo, Y. Chen, N. Xu, and H. Yang, “Energy Efficient Neural Networks for Big Data Analytics,” in *2014 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2014, pp. 1–2.
  - [55] Q. Weng, W. Xiao, Y. Yu, W. Wang, C. Wang, J. He, Y. Li, L. Zhang, W. Lin, and Y. Ding, “MLaaS in the Wild: Workload Analysis and Scheduling in Large-Scale Heterogeneous GPU Clusters,” in *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*, 2022, pp. 945–960.
  - [56] B. Wu, Z. Zhang, Z. Bai, X. Liu, and X. Jin, “Transparent GPU Sharing in Container Clouds for Deep Learning Workloads,” in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, 2023, pp. 69–85.
  - [57] W. Xiao, R. Bhardwaj, R. Ramjee, M. Sivathanu, N. Kwatra, Z. Han, P. Patel, X. Peng, H. Zhao, Q. Zhang *et al.*, “Gandiva: Introspective Cluster Scheduling for Deep Learning,” in *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, 2018, pp. 595–610.
  - [58] W. Xiao, S. Ren, Y. Li, Y. Zhang, P. Hou, Z. Li, Y. Feng, W. Lin, and Y. Jia, “AntMan: Dynamic Scaling on GPU Clusters for Deep Learning,” in *OSDI*, 2020, pp. 533–548.
  - [59] P. Yu and M. Chowdhury, “Fine-grained GPU Sharing Primitives for Deep Learning Applications,” *Proceedings of Machine Learning and Systems*, vol. 2, pp. 98–111, 2020.